Aprendizaje por refuerzo con verificación certificada como única recompensa: aprendizaje verbal con un cuaderno verificado en teoremas de búsqueda de raíces

Nota técnica del proyecto `rl_metodos`

Carlos Manuel Orrego Franco
Universidad Nacional de Colombia, Sede Manizales

Septiembre de 2026 — borrador v0.1, estado al 24 de septiembre de 2026

1 Propósito

Esta nota fija el lenguaje común de los subproyectos de `rl_metodos`. Todos comparten una misma estructura: un *proponente* produce un objeto de prueba y un *juez* independiente decide si lo acepta. La recompensa proviene únicamente del juez; el proponente nunca se evalúa a sí mismo. Lo que varía entre subproyectos es el juez, y con él el residuo de confianza que la aceptación deja sin certificar. Esa variación es deliberada: es el experimento.

Esta nota plantea la pista de *aprendizaje verbal* del subproyecto 09. Un proponente de pesos congelados (`gpt-6-luna`; como segundo modelo, `gpt-6-sol`) intenta demostrar en Isabelle/HOL (Nipkow et al., 2002) una escalera de 48 teoremas sobre cinco métodos de búsqueda de raíces y un grupo de lemas de órdenes de convergencia. Tras cada intento fallido reflexiona sobre los errores del kernel, al estilo de Reflexion (Shinn et al., 2023). Entre episodios consulta un *cuaderno* persistente al que sólo entra lo que el juez verificó. Se actualiza el cuaderno, no los pesos: es aprendizaje en contexto con memoria curada, y así se reporta. Pregunta que responde: con un cuaderno escrito en 32 peldaños de bisección, punto fijo, Newton y órdenes, ¿demuestra el proponente más teoremas de dos métodos que nunca vio (secante y regula falsi, 16 peldaños) que sin el cuaderno?

Cinco principios, acordados antes de escribir código: (i) el kernel de Isabelle es la única fuente de verdad; (ii) el juez, y no el modelo, decide qué entra al cuaderno; (iii) todo enunciado redactado por un modelo pasa por revisión humana antes de usarse; (iv) lo que la automatización resuelve sola no da crédito; (v) todo lo que escribe el modelo se compila aislado, sin red y sin escritura fuera de su directorio.

Esta nota sigue la estructura de la nota técnica: el MDP común (§2), la interfaz de verificador y sus tres instancias (§3), las funciones de recompensa (§4), las métricas (§5), la jerarquía de confianza entre jueces (§6) y los entornos (§7, Tabla 1). Cierra con lo hecho y lo previsto (§8). El Apéndice A trae los enunciados congelados completos.

2 El MDP común

Definición 1 (Entorno de demostración verificada)**.** *Un entorno es una tupla* $\mathcal{E} = (T, \mathcal{A}, V, B)$ *donde* T *es un* objetivo *inmutable (un enunciado y su contexto),* $\mathcal{A}$ *es un espacio de* propuestas*,* V *es un* verificador *y* $B \in \mathbb{N}$ *es un presupuesto de pasos. El MDP asociado es* $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ *con:*

- ***Estado*** $s_t = (T, \mathcal{C}_t, h_t, M)$*: el objetivo, el* contexto observable $\mathcal{C}_t$ *que el juez devuelve tras el último intento (lemas disponibles, obligaciones abiertas con su texto literal, o el valor certificado vigente), y el historial* h_t *de resultados anteriores.*

- ***Cuaderno*** M*: los lemas que el kernel verificó como enunciados independientes, con su procedencia (*`Cuaderno.thy`*); los errores frecuentes, cada uno comprobado mecánicamente (*`errores.md`*); y las estrategias que llevaron a demostraciones aceptadas, con contadores de intentos y éxitos (*`estrategias.md`*). Sólo los episodios de entrenamiento escriben en él; la evaluación usa una copia congelada con su hash. El modelo está congelado: el cuaderno es contexto, no entrenamiento.*

- ***Acción*** $a_t \in \mathcal{A}$*: una propuesta completa (una cadena de lemas, una anotación de invariantes y variantes, o un programa de tamaños de paso). El proponente es un modelo de lenguaje; la acción es su salida tras una extracción determinista.*

- ***Transición*** $P(s_{t+1} \mid s_t, a_t)$*: determinista dado el juez;* s_{t+1} *incorpora el resultado* $V(T, a_t)$ *y actualiza* $\mathcal{C}_{t+1}$ *y* h_{t+1}*. La estocasticidad del sistema vive en el proponente, no en la transición.*

- ***Recompensa*** $R(s_t, a_t) = r(V(T, a_t))$*, función únicamente del resultado del juez (§4).*

- ***Terminación****: el episodio termina en el primer* t *con resultado* ACCEPTED*, o cuando* $t = B$*.*

En la pista verbal la acción y el presupuesto se concretan así:

- Una acción es una sola llamada al modelo, que devuelve una reflexión breve sobre el intento anterior y una demostración Isar completa. La demostración puede traer lemas auxiliares con su demostración y marcar pasos con `by martillo`, que el juez intenta cerrar con automatización (§3).

- Además hay *consultas*, gratuitas porque no cuentan como intento: buscar teoremas por patrón o por nombre, o comprobar si un hecho o un método existe. Se hacen hasta 4 por turno y 8 por episodio.

- El presupuesto es de $B = K = 4$ intentos por episodio. En la comparación entre modelos es $K = 3$, con un turno de consulta. Se registra en qué intento ocurre cada éxito.

Observación 1 (El objetivo es inmutable)**.** *T no forma parte del espacio de acciones. Cualquier propuesta que modifique el enunciado, sus hipótesis o su tipo se rechaza con etiqueta* STATEMENT_DRIFT *(§4.3). Esta restricción se impone en código mediante un hash del enunciado congelado, no por convención. Es la* frontera semántica *del proyecto: el lugar donde la confianza no puede delegarse al juez.*

Observación 2 (Escalera congelada)**.** *Cada peldaño de la escalera lleva su hash congelado, su estado* `aprobado` *y el registro de la aprobación humana (quién y cuándo). El cargador rechaza cualquier peldaño que no esté aprobado: sin aprobación no hay juez.*

3 La interfaz de verificador y sus tres instancias

Definición 2 (Verificador)**.** *Un verificador es una función* $V : \mathcal{T} \times \mathcal{A} \to \mathcal{O}$ *que devuelve un* resultado

$$o = (\texttt{accepted},\ \texttt{label},\ \texttt{partial},\ \texttt{cost_s},\ \texttt{raw}),$$

con $\texttt{accepted} \in \{0, 1\}$*,* $\texttt{label} \in L$ *y* $\texttt{partial} \in [0, 1]$*, donde* L *es un conjunto finito de etiquetas de resultado propio de cada juez,* `partial` *es una medida de progreso que sólo se usa bajo recompensa moldeada, y* `raw` *es la salida íntegra del juez para auditoría.*

3.1 Juez de grafo de lemas (V_G)

Proviene de los subproyectos 01–05: el objetivo es un grafo de lemas escrito a mano, con distractores tomados de errores reales de estudiantes, y la propuesta es una cadena de lemas. En esta pista no se usa como juez. Sirve de fuente: la escalera de la bisección, el punto fijo, Newton y la secante sale de esos grafos, con los mismos pasos y los mismos errores tentadores, ahora como teoremas que hay que demostrar en lugar de nodos que hay que ordenar.

3.2 Juez PEP (V_P)

Proviene del subproyecto 06: certifica cotas de peor caso de programas de tamaños de paso resolviendo un programa semidefinido, con certificado racional exacto (Drori and Teboulle, 2014; Taylor et al., 2017). En esta pista no interviene; se menciona porque es el único juez de la serie independiente del grafo humano además de V_I.

3.3 Juez Isabelle (V_I)

Proviene del subproyecto 08 y del marco de Bryant et al. (2026). En esta pista se usa su **variante de teorema**: el objetivo es un teorema puro de Isabelle/HOL (preludio con las definiciones del método, variables fijadas, hipótesis con nombre y conclusión), congelado y aprobado. La propuesta es el texto Isar de la demostración y, opcionalmente, un bloque de lemas auxiliares con su demostración. El juez procede así.

1. Filtro léxico: sólo texto de demostración. Nada de ML, `sorry`, `oops`, axiomas, definiciones ni nombres reservados, que son las etiquetas de las hipótesis, las constantes del preludio y `isarl*`.

2. Compila el peldaño con el texto del agente dentro de un contexto con las hipótesis. Comprueba que el enunciado leído antes y después del texto del agente coincide término a término.

3. Exporta el teorema al contexto global instanciando por nombre cada variable, con `isarl_target[where v = v, OF assms]`, para que sólo dependa de las hipótesis congeladas.

4. Comprueba en el kernel que no dependa de oráculos ni de `skip_proof` y que no haya axiomas nuevos.

Cada compilación corre bajo `sandbox-exec`, sin red externa ni escritura fuera de su directorio, y cada veredicto guarda la versión de Isabelle, la del juez, el límite de tiempo, la carga de la máquina y el hash del objetivo. Etiquetas: ACCEPTED (compila sin oráculos, sin `skip_proof` ni axiomas nuevos y el teorema es el enunciado congelado), PROOF_FAILED (errores en la demostración del teorema), LEMMA_FAILED (errores en los lemas del agente), UNSOUND (oráculos, `skip_proof`, axiomas nuevos, o `sorry`/`oops` en el texto), STATEMENT_DRIFT (el teorema probado no es el congelado, el objetivo cambió en disco o hay comandos que cambian el significado), SYNTAX_ERROR (cadena o comentario sin cerrar), TIMEOUT (se agota el tiempo de compilación) y INVALID_PROPOSAL (comandos o nombres no admitidos).

Martillo y consulta. El martillo es `try0` (5 s) y, si no cierra, Sledgehammer (60 s) (Paulson and Blanchette, 2010). Se usa de dos maneras.

- Solo, sobre la meta del peldaño: es la configuración A, la línea base.

- Sobre cada paso que el agente marca con `by martillo`. Una primera pasada busca una línea para cada marca (caché, `try0` o Sledgehammer) y sigue adelante. Luego se pegan las líneas y el juez compila la demostración completa sin atajos.

Ninguna línea cuenta sin el kernel. Las llamadas al martillo van en serie, porque su límite es de tiempo real, y se guardan en una caché cuya clave es el enunciado, el cuaderno, los límites y la

versión de Isabelle. La consulta lee el patrón del agente como dato, nunca como código, y lo pasa a `find_theorems` en el mismo contexto que el juez; devuelve a lo sumo 20 teoremas.

Glosario. Los modelos escriben Lean con frecuencia. El glosario tiene 25 entradas de Lean a Isabelle, sacadas de los errores de las corridas anteriores. Se comprobó en Isabelle2025 (todas pasan): cada nombre de Lean que desaconseja no existe y cada alternativa que recomienda sí existe. Así se siembra `errores.md`.

Observación 3 (Existencia frente a descubrimiento)**.** *En la línea base (completa), el martillo solo resuelve 1 de 48 peldaños, con la línea aceptada por el juez:* `pf_3_unicidad`*. En los 47 que no cierra, Sledgehammer agota sus 60 s sin encontrar nada. Los peldaños con definiciones recursivas (la bisección, las iteraciones) piden inducción, y Sledgehammer no hace inducción. En cambio, en* `orden_1_lineal`*, con el esqueleto de la inducción escrito y el paso inductivo marcado* `by martillo`*, Sledgehammer encuentra la línea* `using ordered_comm_semiring_class.comm_mult_left_mono by blast` *y el juez acepta la demostración completa: 21 s en total, búsqueda y verificación incluidas, y 15 s la segunda vez, con la caché. Por eso la recompensa (§4) no premia lo que el martillo hace solo: el mérito está en el esqueleto.*

4 Funciones de recompensa

4.1 Recompensa base (todos los jueces)

$$r(o) = \begin{cases} 1 & \text{si } o.\texttt{accepted} = 1, \\ 0 & \text{en otro caso.} \end{cases}$$

Es la única recompensa activa por defecto. Es binaria a propósito: cualquier señal intermedia introduce un juicio sobre qué es "progreso", y ese juicio debe declararse, no esconderse.

4.2 Recompensa moldeada (opcional, tras bandera)

$$r_\beta(o) = r(o) + \beta \cdot o.\texttt{partial} \cdot \mathbb{1}[o.\texttt{accepted} = 0], \qquad \beta \in [0, 0.5].$$

Para V_P se conserva la recompensa original del subproyecto 06, que es continua por naturaleza: $r_P(o) = \max\{-2,\ \log(\tau_{\text{ref}}/\tau(h))\}$ si el programa es válido y -3 en otro caso, donde τ_{ref} es la mejor cota certificada conocida. Se reporta también su binarización $\mathbb{1}[\tau(h) < \tau_{\text{ref}}]$ para hacerla comparable con los otros jueces.

4.3 Penalización por deriva del enunciado

STATEMENT_DRIFT recibe recompensa 0, no negativa, en la configuración base. La razón es metodológica: queremos *contar* cuántas veces el proponente intenta cambiar la pregunta, y una penalización alteraría la frecuencia que se pretende medir. Una variante con penalización $-\delta$ queda disponible tras bandera para experimentos de control.

4.4 En la pista verbal

Los pesos no se actualizan, así que la recompensa no entrena nada: es una *métrica*. Para un peldaño T y un intento con resultado o,

$$r_V(o) = \rho(o) + b \cdot \mathbb{1}[o.\texttt{accepted} = 1],$$

salvo en cuatro casos, en los que $r_V(o) = 0$:

- el martillo solo resuelve el peldaño ($T \in \mathcal{M}$);
- la prueba del agente es sólo `by martillo` sobre la meta;

- la prueba aceptada es una sola línea opaca;
- la etiqueta es STATEMENT_DRIFT, UNSOUND, SYNTAX_ERROR o INVALID_PROPOSAL.

Aquí $b = 1$. $\mathcal{M}$ son los peldaños que el martillo resuelve solo con el mismo cuaderno: sin esa línea base no se calcula r_V. $\rho(o) = 1$ si la prueba se aceptó y, si no, la fracción de pasos `by martillo` que el martillo cerró (0 sin marcas). Una línea es *opaca* si usa `smt` o `metis` con más de 4 hechos, contando los de su `using`. Una prueba es *una sola línea opaca* si, quitando los cierres triviales (`.`, `..`, `by simp`, `by auto` y similares, sin argumentos), le queda una única invocación y es opaca. La regla, aprobada por el autor el 24 de septiembre, no cambia lo que acepta el kernel: sólo niega el crédito, porque eso es «el martillo lo hizo». Lo que entra al cuaderno se decide con reglas que no dependen de r_V:

- **Lemas**: sólo si el kernel los verifica como enunciados independientes. Se compilan a nivel de teoría, fuera del contexto con las hipótesis del peldaño, con el cuaderno previo como único contexto extra. Un lema que usa constantes del preludio de un método es de ese método; si no usa ninguna, es general. No puede tapar un hecho que ya exista.
- **Errores**: de cualquier episodio de entrenamiento. Si afirman que un nombre o método existe o no, se comprueban mecánicamente.
- **Estrategias**: sólo de episodios aceptados o que aportaron lemas verificados. Llevan contadores de intentos y éxitos, y se descartan si tras 5 usos su tasa de éxito es menor que 0,2.

La recompensa por lemas parciales (crédito a lemas que el kernel acepta aunque la demostración falle) queda apagada: pertenece a la pista de programas y no se prueba ni se enciende en esta.

5 Métricas

Para cada entorno, juez y proponente se registran:

1. **Intentos hasta aceptación**: $t^* \leq K$ por episodio, censurado en K, y en qué intento ocurre cada éxito.
2. **Distribución de etiquetas**: conteo de cada etiqueta de V_I sobre todos los intentos. SYNTAX_ERROR se cuenta aparte desde la Etapa 2; antes se confundía con la deriva.
3. **Razón de deriva**: STATEMENT_DRIFT frente a PROOF_FAILED.
4. **Transferencia**: es la medida central de esta pista. Se mide la tasa de éxito en los peldaños de test (métodos que no aparecen en entrenamiento) con el cuaderno congelado (configuración E) frente a sin él (C). La comparación es pareada por peldaño, con conteo de pares discordantes y prueba exacta de McNemar. Se reporta por teorema y por nivel de dificultad; con sólo dos métodos en test no se remuestrea por método. `secante_6c_orden_phi` se interpreta junto a `newton_6c_q_orden2`, su análogo directo en entrenamiento.
5. **Comparación**: en lugar de comparar jueces, se comparan configuraciones (A, B0, B, C, D, E, F; §8) y modelos (Luna frente a Sol, pareados por peldaño y configuración, con el mismo presupuesto).
6. **Costo**: dólares reales de cada llamada según sus tokens (libro de gastos), segundos de modelo, de juez y de martillo, y consultas usadas.

Además, cada prueba aceptada lleva una medida mecánica de *legibilidad*, que se reporta junto a la tasa de éxito:

- pasos explícitos (`have`, `show`, `obtain`);
- invocaciones de métodos;
- líneas opacas;
- pasos cerrados por el martillo;
- comentarios, líneas y caracteres.

Es la respuesta al pedido del autor de que las demostraciones sean inteligibles. La prueba de referencia de `orden_1_lineal`, por ejemplo, tiene 4 pasos explícitos, 3 invocaciones no triviales y ninguna línea opaca.

Aún no hay corridas con modelo en esta pista: el libro de gastos registra 0 llamadas y $0. Los únicos valores medidos son los del juez y el martillo (§3 y §8).

6 Jerarquía de confianza entre jueces

Los tres jueces aceptan objetos distintos y dejan residuos de confianza distintos. Ordenados de menor a mayor independencia respecto del humano:

juez	qué certifica	en qué reposa la confianza
V_G	validez de una cadena en un grafo dado	el grafo, escrito por el profesor
V_P	una cota numérica de peor caso	el SDP, la certificación racional exacta
V_I	un teorema en el kernel de Isabelle/HOL	el kernel; la adecuación del triple congelado

Ningún juez elimina la confianza; cada uno la desplaza. V_G la desplaza al autor del grafo. V_P la desplaza a la formulación del PEP y a la implementación del certificador. V_I la desplaza a dos lugares que el kernel no puede inspeccionar: la correspondencia entre el triple de Hoare y el enunciado informal que se pretendía probar, y la matemática auxiliar importada. Al reproducir la bisección de Bryant et al. (2026) en el subproyecto 08 se vio que incluso lo que el artículo reporta como "automático" contiene decisiones humanas en esa frontera. El objetivo del proyecto es medir ese residuo, no negarlo.

Residuo en esta pista: la adecuación de cada enunciado congelado al teorema que se quería. Los redactó Claude a partir de los grafos de 01–05 y de la literatura clásica, y el autor los revisó y aprobó uno por uno: esa revisión es la frontera semántica. Para acotarla, 11 peldaños quedaron marcados como versiones debilitadas y cada uno tiene su versión clásica en la escalera. 9 peldaños quedaron marcados porque arrastran hipótesis que no usan. Además hubo tres comprobaciones mecánicas. Isabelle leyó y tipó las hipótesis y la conclusión de cada método (6 de 6 métodos). Una interpretación con valores concretos mostró que las hipótesis no son contradictorias. Una comprobación numérica ejecutó el método con problemas aleatorios y confirmó la conclusión en todos los casos (48 de 48 peldaños con comprobación). La teoría final de cada peldaño exporta el teorema y detecta un `sorry` de prueba como `skip_proof` en 48 de 48 peldaños. Se importa HOL-Analysis como matemática de confianza.

7 Entornos

El entorno de esta pista es la escalera `corpus/raices`: peldaños pequeños por método, del paso elemental al teorema de convergencia con su orden. La separación entre entrenamiento y test es por método completo, peldaños bajos incluidos. Así el test mide transferencia a métodos no vistos y no a variantes de un mismo método.

split	dif. 1	dif. 2	dif. 3	dif. 4	dif. 5	total
train	3	12	10	6	1	32
test	0	5	3	3	5	16

método	split	peldaños	clásicos	debilitados	dificultad	resueltos por el martillo
bisección	train	8	7	1	1–4	0 de 8
punto fijo	train	12	10	2	1–4	1 de 12
Newton	train	9	5	4	2–5	0 de 9
órdenes de convergencia	train	3	3	0	1–2	0 de 3
secante	test	9	6	3	2–5	0 de 9
regula falsi	test	7	6	1	2–5	0 de 7
total train		32	25	7		1 de 32
total test		16	12	4		0 de 16

Table 1: Escalera de búsqueda de raíces. Un peldaño debilitado añade hipótesis o fija constantes respecto del enunciado clásico; cada uno tiene su versión clásica en la escalera. La dificultad va de 1 (un paso) a 5 (orden de convergencia clásico). La última columna es la línea base de la configuración A (§3); sin línea base, un guion. El proponente es `gpt-6-luna` con esfuerzo alto; `gpt-6-sol`, sólo en la comparación pareada sobre test.

8 Experimentos realizados y previstos

Hecho hasta el 24 de septiembre de 2026:

1. **Etapa 0, plan.** Inventario del código, del juez y de los costos medidos, y plan por etapas. El autor lo aprobó con ajustes: glosario y consulta en el entorno base, teoremas puros primero, test por método y presupuesto con tope en dólares.

2. **Etapa 1, escalera.** 48 peldaños (32 de entrenamiento y 16 de test), todos aprobados por el autor. Validación en Isabelle de 6 de 6 métodos; exportación y detección de `sorry` en 48 de 48 peldaños.

3. **Etapa 2, juez robusto.** Juez por peldaño, etiqueta SYNTAX_ERROR, exportación con instanciación por nombre, aislamiento con `sandbox-exec` (escribir fuera y la red externa quedan bloqueados), metadatos en cada veredicto y libro de gastos con tope en dólares.

4. **Etapa 3, martillo.** Línea base de la configuración A: 48 de 48 peldaños medidos, 1 resuelto por el martillo solo, 76 s de media por peldaño. Cuatro peldaños se repitieron porque el Mac se suspendió durante la búsqueda: el límite de Sledgehammer corre en tiempo de pared y la suspensión lo consumía. Ahora el martillo compara el reloj de pared con el monótono, descarta y repite toda búsqueda con suspensión, y las corridas van bajo `caffeinate`. Martillo por pasos comprobado en `orden_1_lineal` (Observación 3). Glosario comprobado (25 entradas) y errores del cuaderno sembrados desde él. Consulta de teoremas con saneado y tope.

5. **Etapa 4, recompensa y admisión.** Se implementaron r_V con la regla de la línea opaca, la legibilidad y las reglas de admisión al cuaderno. Se comprobaron en Isabelle: entran un lema general y uno de bisección con su preludio. Se rechazan tres: uno falso, uno que usaba la hipótesis `ab` del peldaño y uno que se llamaba `mult_left_mono` (taparía el de Isabelle).

6. **Pruebas automáticas.** Rápidas: 148 de 148 pasan. Con Isabelle: 17 de 17 pasan.

Previsto:

1. **Etapa 5**: el bucle de reflexión con K intentos, consultas en lotes y topes de llamadas por fase. La instrucción al proponente pide demostraciones inteligibles: Isar estructurado, un paso por idea, hipótesis citadas por su nombre y un comentario breve en español en cada paso no trivial. Cada prueba aceptada irá completa en el informe final.

2. **Etapa 6**: el cuaderno (`Cuaderno.thy`, `errores.md`, `estrategias.md`), con admisión por el juez; sólo lo escribe el entrenamiento.

3. **Etapa 7**: los experimentos. Configuraciones: A (martillo solo); B0 (proponente y martillo, sin glosario, consulta ni reintentos); B (B0 con glosario y consulta); C (B con reintentos y reflexión); D (C con el cuaderno de errores); E (D con los lemas); F (E con las estrategias). Primera ronda: A, B0, B, C y E; luego Sol en C y E sobre test.

fase	llamadas	con 20 %	costo
cuaderno (Luna, train)	197	236	\$1,54
evaluación completa (Luna, test, 2 episodios)	512	614	\$3,99
comparación pareada (Luna, C y E, $K = 3$)	128	154	\$1,00
comparación pareada (Sol, C y E, $K = 3$)	128	154	\$19,97
total			\$26,50

Costos estimados con 5 000 tokens de entrada y 12 000 de salida por llamada, a precios de lista. El tope del libro de gastos es \$25; sube a \$28 sólo si el costo medido de Sol lo exige, y antes de usar Sol se miden 3 llamadas. La evaluación de Luna conserva sus 2 episodios por peldaño, que miden la variabilidad dentro de un mismo modelo.

References

Bryant, D., Huerta y Munive, J. J., and Foster, S. (2026). Verifying numerical methods with Isabelle/HOL. v3, 21 Sep 2026; código: github.com/isabelle-utp/Verified_Numerical_Methods.

Drori, Y. and Teboulle, M. (2014). Performance of first-order methods for smooth convex minimization: A novel approach. *Mathematical Programming*, 145(1–2):451–482.

Nipkow, T., Paulson, L. C., and Wenzel, M. (2002). *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer.

Paulson, L. C. and Blanchette, J. C. (2010). Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics (IWIL 2010)*.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Taylor, A. B., Hendrickx, J. M., and Glineur, F. (2017). Smooth strongly convex interpolation and exact worst-case performance of first-order methods. *Mathematical Programming*, 161(1–2):307–345.

A La escalera completa

Enunciados congelados y aprobados, en la sintaxis ASCII de Isabelle (`<=` es $\leq$, `|x|` es el valor absoluto, `!!` y `ALL` son cuantificadores universales, `%x.` es $\lambda x.$). Cada peldaño da su identificador, su split, su dificultad, su versión y el enunciado en español con el que se aprobó.

A.1 Bisección (train)

Preludio: `primrec biseccion :: "(real => real) => real => real => nat => real * real" where "biseccion f a b 0 = (a, b)" | "biseccion f a b (Suc n) = (case biseccion f a b n of (l, u) => (if f l * f ((l + u) / 2) <= 0 then (l, (l + u) / 2) else ((l + u) / 2, u)))" definition punto_ medio :: "(real => real) => real => real => nat => real" where "punto_medio f a b n = (fst (biseccion f a b n) + snd (biseccion f a b n)) / 2"`

- `bis_1_ancho` (train, dificultad 1, clásica; arrastra hipótesis que no usa: a $\leq$ b). Tras n pasos, el intervalo mide $(b - a)/2^n$.
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `ab: "a <= b"` **shows** `snd (biseccion f a b n) - fst (biseccion f a b n) = (b - a) / 2 ^ n`

- `bis_2_encaje` (train, dificultad 2, clásica). Los intervalos están contenidos en [a, b], son no vacíos y encajados: $a_n \leq a_{n+1}$ y $b_{n+1} \leq b_n$.
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `ab: "a <= b"` **shows** `a <= fst (biseccion f a b n) & fst (biseccion f a b n) <= snd (biseccion f a b n) & snd (biseccion f a b n) <= b & fst (biseccion f a b n) <= fst (biseccion f a b (Suc n)) & snd (biseccion f a b (Suc n)) <= snd (biseccion f a b n)`

- `bis_3_signo` (train, dificultad 2, clásica). Invariante de signo: si f(a)·f(b) ≤ 0, entonces f(a_n)·f(b_n) ≤ 0 para todo n.
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `signo: "f a * f b <= 0"` **shows** `f (fst (biseccion f a b n)) * f (snd (biseccion f a b n)) <= 0`

- `bis_4_raiz` (train, dificultad 3, clásica). Con f continua y f(a)·f(b) ≤ 0, cada intervalo [a_n, b_n] contiene una raíz (Bolzano).
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `ab: "a <= b" and cont: "continuous_on {a..b} f" and signo: "f a * f b <= 0"` **shows** `EX c. fst (biseccion f a b n) <= c & c <= snd (biseccion f a b n) & f c = 0`

- `bis_5_cota` (train, dificultad 3, debilitada; clásica: `bis_5c_cota_uniforme`). Existe una raíz c en [a, b] con $|m_n - c| \leq (b - a)/2^{n+1}$ (la cota del teorema de 01).
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `ab: "a <= b" and cont: "continuous_on {a..b} f" and signo: "f a * f b <= 0"` **shows** `EX c. a <= c & c <= b & f c = 0 & |punto_medio f a b n - c| <= (b - a) / 2 ^ Suc n`

- `bis_5c_cota_uniforme` (train, dificultad 3, clásica). Existe una raíz c en [a, b] tal que, para todo n, $|m_n - c| \leq (b - a)/2^{n+1}$ (la misma c para todo n).
  fixes `f :: "real => real" and a b :: real` **assumes** `ab: "a <= b" and cont: "continuous_on {a..b} f" and signo: "f a * f b <= 0"` **shows** `EX c. a <= c & c <= b & f c = 0 & (ALL n. |punto_medio f a b n - c| <= (b - a) / 2 ^ Suc n)`

- `bis_6_iteraciones` (train, dificultad 2, clásica). Si n $\geq \log_2((b - a)/\text{tol})$, el intervalo mide a lo sumo tol.
  fixes `f :: "real => real" and a b :: real and tol :: real and n :: nat` **assumes** `ab: "a < b" and tol: "0 < tol"` **shows** `log 2 ((b - a) / tol) <= real n ==> snd (biseccion f a b n) - fst (biseccion f a b n) <= tol`

- `bis_7_convergencia` (train, dificultad 4, clásica). Los puntos medios convergen a una raíz c de f en [a, b].
  fixes `f :: "real => real" and a b :: real` **assumes** `ab: "a <= b" and cont: "continuous_on {a..b} f" and signo: "f a * f b <= 0"` **shows** `EX c. a <= c & c <= b & f c = 0 & (%k. punto_medio f a b k) ----> c`

A.2 Punto fijo (train)

- `pf_1_derivada` (train, dificultad 2, clásica). Si |g'| $\leq$ k en [a, b], g es k-Lipschitz en [a, b] (valor medio).
  fixes `g g' :: "real => real" and a b k x y :: real` **assumes** `ab: "a <= b" and deriv: "!!t. t : {a..b} ==> (g has_real_derivative g' t) (at t)" and cota: "!!t. t : {a..b} ==> |g' t| <= k" and hx: "x : {a..b}" and hy: "y : {a..b}"` **shows** `|g x - g y| <= k * |x - y|`

- `pf_2_dentro` (train, dificultad 1, clásica; arrastra hipótesis que no usa: la contracción (contr) y 0 $\leq$ k $<$ 1). Los iterados se quedan en [a, b].
  fixes `g :: "real => real" and a b k x0 :: real and n :: nat` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}"` **shows** `(g ^^ n) x0 : {a..b}`

- `pf_3_unicidad` (train, dificultad 2, clásica; arrastra hipótesis que no usa: x0 y la invariancia de [a, b] (dentro)). Hay a lo sumo un punto fijo en [a, b].
  fixes `g :: "real => real" and a b k x0 :: real and p q :: real` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}" and hp: "p : {a..b}" and fp: "g p = p" and hq: "q : {a..b}" and fq: "g q = q"` **shows** `p = q`

- `pf_4_decaimiento` (train, dificultad 2, clásica). $|x_n - p| \leq k^n |x0 - p|$.
  fixes `g :: "real => real" and a b k x0 :: real and p :: real and n :: nat` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}" and hp: "p : {a..b}" and fp: "g p = p"` **shows** `|(g ^^ n) x0 - p| <= k ^ n * |x0 - p|`

- `pf_5_a_priori` (train, dificultad 2, debilitada; clásica: `pf_5c_a_priori_max`). Cota a priori: $|x_n - p| \leq k^n (b - a)$.
  fixes `g :: "real => real" and a b k x0 :: real and p :: real and n :: nat` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}" and hp: "p : {a..b}" and fp: "g p = p"` **shows** `|(g ^^ n) x0 - p| <= k ^ n * (b - a)`

- `pf_5c_a_priori_max` (train, dificultad 2, clásica). Cota a priori clásica: $|x_n - p| \leq k^n \cdot \max(x0 - a, b - x0)$.
  fixes `g :: "real => real" and a b k x0 :: real and p :: real and n :: nat` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}" and hp: "p : {a..b}" and fp: "g p = p"` **shows** `|(g ^^ n) x0 - p| <= k ^ n * max (x0 - a) (b - x0)`

- `pf_6_a_priori_x1` (train, dificultad 3, clásica). Cota a priori en función del primer paso: $|x_n - p| \leq k^n/(1 - k) \cdot |x1 - x0|$.
  fixes `g :: "real => real" and a b k x0 :: real and p :: real and n :: nat` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}" and hp: "p : {a..b}" and fp: "g p = p"` **shows** `|(g ^^ n) x0 - p| <= k ^ n / (1 - k) * |g x0 - x0|`

- `pf_6p_a_posteriori` (train, dificultad 3, clásica). Cota a posteriori (terminología usual): $|x_n - p| \leq k/(1 - k) \cdot |x_n - x_{n-1}|$ para $n \geq 1$.
  fixes `g :: "real => real" and a b k x0 :: real and p :: real and n :: nat` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}" and hp: "p : {a..b}" and fp: "g p = p"` **shows** `|(g ^^ Suc n) x0 - p| <= k / (1 - k) * |(g ^^ Suc n) x0 - (g ^^ n) x0|`

- `pf_7_existencia` (train, dificultad 3, clásica; arrastra hipótesis que no usa: x0). g tiene un punto fijo en [a, b].
  fixes `g :: "real => real" and a b k x0 :: real` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}"` **shows** `EX p : {a..b}. g p = p`

- `pf_8_convergencia` (train, dificultad 4, clásica). La iteración converge al punto fijo.
  fixes `g :: "real => real" and a b k x0 :: real` **assumes** `ab: "a <= b" and dentro: "!!x. x : {a..b} ==> g x : {a..b}" and contr: "!!x y. x : {a..b} ==> y : {a..b} ==> |g x - g y| <= k * |x - y|" and k0: "0 <= k" and k1: "k < 1" and x0: "x0 : {a..b}"` **shows** `EX p : {a..b}. g p = p & (%n. (g ^^ n) x0) ----> p`

- `pf_9_cuadratico` (train, dificultad 3, debilitada; clásica: `pf_9c_cuadratico_medio`). Si $|g'(t)| \leq L|t - p|$ (en particular $g'(p) = 0$), un paso es cuadrático: $|g(x) - p| \leq L(x - p)^2$.

fixes `g g' :: "real => real" and a b p L x :: real` **assumes** `ab: "a <= b" and hp: "p : {a..b}" and fp: "g p = p" and deriv: "!!t. t : {a..b} ==> (g has_real_derivative g' t) (at t)" and cota: "!!t. t : {a..b} ==> |g' t| <= L * |t - p|" and hx: "x : {a..b}"` **shows** `|g x - p| <= L * (x - p)^2`

- `pf_9c_cuadratico_medio` (train, dificultad 3, clásica). Con la constante clásica: $|g(x) - p| \leq (L/2)(x - p)^2$.
  fixes `g g' :: "real => real" and a b p L x :: real` **assumes** `ab: "a <= b" and hp: "p : {a..b}" and fp: "g p = p" and deriv: "!!t. t : {a..b} ==> (g has_real_derivative g' t) (at t)" and cota: "!!t. t : {a..b} ==> |g' t| <= L * |t - p|" and hx: "x : {a..b}"` **shows** `|g x - p| <= L / 2 * (x - p)^2`

A.3 Newton (train)

Preludio: `definition paso_newton :: "(real => real) => (real => real) => real => real" where "paso_newton f f' x = x - f x / f' x"`

- `newton_1_paso` (train, dificultad 4, clásica; arrastra hipótesis que no usa: $M \cdot \delta \leq m$). Error de un paso: $|N(x) - r| \leq M/(2m) \cdot (x - r)^2$ (Taylor con resto de Lagrange).
  fixes `f f' f'' :: "real => real" and r delta m M :: real and x :: real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx: "|x - r| <= delta"` **shows** `|paso_newton f f' x - r| <= M / (2 * m) * (x - r)^2`

- `newton_2_contraccion` (train, dificultad 2, debilitada; clásica: `newton_2c_contraccion_local`). Con $M \cdot \delta \leq m$, un paso reduce el error al menos a la mitad (y el iterado se queda en I).
  fixes `f f' f'' :: "real => real" and r delta m M :: real and x :: real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx: "|x - r| <= delta"` **shows** `|paso_newton f f' x - r| <= |x - r| / 2`

- `newton_2c_contraccion_local` (train, dificultad 4, clásica). Forma clásica local: existe $\delta > 0$ tal que un paso de Newton reduce el error al menos a la mitad en $[r - \delta, r + \delta]$.
  fixes `f f' f'' :: "real => real" and r epsilon :: real` **assumes** `eps: "0 < epsilon" and d1: "!!t. |t - r| < epsilon ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| < epsilon ==> (f' has_real_derivative f'' t) (at t)" and c2: "continuous_on {r - epsilon<..<r + epsilon} f''" and raiz: "f r = 0" and simple: "f' r ~= 0"` **shows** `EX delta>0. ALL x. |x - r| <= delta --> |paso_newton f f' x - r| <= |x - r| / 2`

- `newton_3_decaimiento` (train, dificultad 2, debilitada; clásica: `newton_4c_convergencia_local`). $|x_n - r| \leq \delta/2^n$.
  fixes `f f' f'' :: "real => real" and r delta m M :: real and x0 :: real and n :: nat` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx0: "|x0 - r| <= delta"` **shows** `|(paso_newton f f' ^^ n) x0 - r| <= delta / 2 ^ n`

- `newton_4_convergencia` (train, dificultad 3, debilitada; clásica: `newton_4c_convergencia_local`). Newton converge a r desde todo $x0 \in I$.
  fixes `f f' f'' :: "real => real" and r delta m M :: real and x0 :: real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx0: "|x0 - r| <= delta"` **shows** `(%n. (paso_newton f f' ^^ n) x0) ----> r`

- `newton_4c_convergencia_local` (train, dificultad 4, clásica). Teorema clásico de convergencia local de Newton: existe $\delta > 0$ tal que Newton converge a r desde todo x0 $\in$ [r $- \delta$, r $+ \delta$].
  fixes `f f' f'' :: "real => real" and r epsilon :: real` **assumes** `eps: "0 < epsilon" and d1: "!!t. |t - r| < epsilon ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| < epsilon ==> (f' has_real_derivative f'' t) (at t)" and c2: "continuous_on {r - epsilon<..<r + epsilon} f''" and raiz: "f r = 0" and simple: "f' r ~= 0"` **shows** `EX delta>0. ALL x0. |x0 - r| <= delta --> (%n. (paso_newton f f' ^^ n) x0) ----> r`

- `newton_5_orden2` (train, dificultad 3, debilitada; clásica: `newton_5c_orden2_local`). Orden 2: con $\varepsilon_n = \mathrm{M}/(2\mathrm{m})|\mathrm{x}_n - \mathrm{r}|$, $\varepsilon_n \leq \varepsilon_0^{(2^n)}$.
  fixes `f f' f'' :: "real => real" and r delta m M :: real and x0 :: real and n :: nat` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx0: "|x0 - r| <= delta"` **shows** `M / (2 * m) * |(paso_newton f f' ^^ n) x0 - r| <= (M / (2 * m) * |x0 - r|) ^ (2 ^ n)`

- `newton_5c_orden2_local` (train, dificultad 4, clásica). Orden 2 clásico: existen $\delta > 0$ y C con $|\mathrm{x}_{n+1} - \mathrm{r}| \leq \mathrm{C}(\mathrm{x}_n - \mathrm{r})^2$ para todo x0 $\in [\mathrm{r} - \delta, \mathrm{r} + \delta]$ y todo n.
  fixes `f f' f'' :: "real => real" and r epsilon :: real` **assumes** `eps: "0 < epsilon" and d1: "!!t. |t - r| < epsilon ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| < epsilon ==> (f' has_real_derivative f'' t) (at t)" and c2: "continuous_on {r - epsilon<..<r + epsilon} f''" and raiz: "f r = 0" and simple: "f' r ~= 0"` **shows** `EX delta>0. EX C. ALL x0 n. |x0 - r| <= delta --> |(paso_newton f f' ^^ Suc n) x0 - r| <= C * ((paso_newton f f' ^^ n) x0 - r)^2`

- `newton_6c_q_orden2` (train, dificultad 5, clásica; análogo: `secante_6c_orden_phi`). Q-orden 2 clásico: si ningún iterado cae en r, $|\mathrm{e}_{n+1}|/\mathrm{e}_n^2 \to |\mathrm{f}''(\mathrm{r})/(2\mathrm{f}'(\mathrm{r}))|$ (análogo de secante_6c; si f''(r) = 0 el límite es 0).
  fixes `f f' f'' :: "real => real" and r epsilon :: real` **assumes** `eps: "0 < epsilon" and d1: "!!t. |t - r| < epsilon ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| < epsilon ==> (f' has_real_derivative f'' t) (at t)" and c2: "continuous_on {r - epsilon<..<r + epsilon} f''" and raiz: "f r = 0" and simple: "f' r ~= 0"` **shows** `EX delta>0. ALL x0. |x0 - r| <= delta --> (ALL n. (paso_newton f f' ^^ n) x0 ~= r) --> (%n. |(paso_newton f f' ^^ Suc n) x0 - r| / ((paso_newton f f' ^^ n) x0 - r)^2) ----> |f'' r / (2 * f' r)|`

A.4 Órdenes de convergencia (train)

- `orden_1_lineal` (train, dificultad 1, clásica). Recurrencia lineal: $|\mathrm{e}_n| \leq \mathrm{c}^n|\mathrm{e}_0|$.
  fixes `e :: "nat => real" and c :: real and n :: nat` **assumes** `c0: "0 <= c" and paso: "!!j. |e (Suc j)| <= c * |e j|"` **shows** `|e n| <= c ^ n * |e 0|`

- `orden_2_cuadratico_lineal` (train, dificultad 2, clásica). Si $|\mathrm{e}_{n+1}| \leq \mathrm{C}\ \mathrm{e}_n^2$ y $\mathrm{C}|\mathrm{e}_0| \leq 1/2$, entonces $|\mathrm{e}_n| \leq |\mathrm{e}_0|/2^n$.
  fixes `e :: "nat => real" and C :: real and n :: nat` **assumes** `C0: "0 <= C" and paso: "!!j. |e (Suc j)| <= C * (e j)^2" and inicio: "C * |e 0| <= 1 / 2"` **shows** `|e n| <= |e 0| / 2 ^ n`

- `orden_3_cuadratico` (train, dificultad 2, clásica). Si $|\mathrm{e}_{n+1}| \leq \mathrm{C}\ \mathrm{e}_n^2$, entonces $\mathrm{C}|\mathrm{e}_n| \leq (\mathrm{C}|\mathrm{e}_0|)^{(2^n)}$.
  fixes `e :: "nat => real" and C :: real and n :: nat` **assumes** `C0: "0 <= C" and paso: "!!j. |e (Suc j)| <= C * (e j)^2"` **shows** `C * |e n| <= (C * |e 0|) ^ (2 ^ n)`

A.5 Secante (test)

Preludio: `definition paso_secante :: "(real => real) => real => real => real" where "paso_secante f x y = y - f y * (y - x) / (f y - f x)" fun secante :: "(real => real) => real => real => nat => real" where "secante f x0 x1 0 = x0" | "secante f x0 x1 (Suc 0) = x1" | "secante f x0 x1 (Suc (`

```
Suc n)) = paso_secante f (secante f x0 x1 n) (secante f x0 x1 (Suc n))" fun fibo ::  "nat => nat"
where "fibo 0 = 0" | "fibo (Suc 0) = 1" | "fibo (Suc (Suc n)) = fibo (Suc n) + fibo n"
```

- `secante_1_inyectiva` (test, dificultad 2, clásica; arrastra hipótesis que no usa: f'' y su cota, M·$\delta \leq$ m). En I, f es inyectiva (|f'| $\geq$ m $>$ 0): el paso de la secante está bien definido.
  fixes `f f' f'' ::  "real => real" and r delta m M ::  real and x y ::  real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx: "|x - r| <= delta" and hy: "|y - r| <= delta" and dist: "x ~= y"` **shows** `f x ~= f y`

- `secante_2_cociente` (test, dificultad 2, clásica; arrastra hipótesis que no usa: f'' y su cota, M·$\delta \leq$ m). El cociente de diferencias en I está acotado por abajo por m.
  fixes `f f' f'' ::  "real => real" and r delta m M ::  real and x y ::  real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx: "|x - r| <= delta" and hy: "|y - r| <= delta" and dist: "x ~= y"` **shows** `m <= |(f y - f x) / (y - x)|`

- `secante_3_producto` (test, dificultad 5, clásica; arrastra hipótesis que no usa: M·$\delta \leq$ m). Cota producto: |S(x, y) − r| $\leq$ M/(2m)·|x − r| · |y − r| (diferencias divididas).
  fixes `f f' f'' ::  "real => real" and r delta m M ::  real and x y ::  real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx: "|x - r| <= delta" and hy: "|y - r| <= delta" and dist: "x ~= y"` **shows** `|paso_secante f x y - r| <= M / (2 * m) * |x - r| * |y - r|`

- `secante_4_contraccion` (test, dificultad 2, debilitada; clásica: `secante_4c_contraccion_local`). Con M·$\delta \leq$ m, un paso de la secante reduce el error del iterado actual al menos a la mitad.
  fixes `f f' f'' ::  "real => real" and r delta m M ::  real and x y ::  real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx: "|x - r| <= delta" and hy: "|y - r| <= delta" and dist: "x ~= y"` **shows** `|paso_secante f x y - r| <= |y - r| / 2`

- `secante_4c_contraccion_local` (test, dificultad 4, clásica). Forma clásica local: existe $\delta > 0$ tal que un paso de la secante con x $\neq$ y en [r − δ, r + δ] reduce a la mitad el error del iterado actual.
  fixes `f f' f'' ::  "real => real" and r epsilon ::  real` **assumes** `eps:  "0 < epsilon" and d1: "!!t.  |t - r| < epsilon ==> (f has_real_derivative f' t) (at t)" and d2:  "!!t.  |t - r| < epsilon ==> (f' has_real_derivative f'' t) (at t)" and c2:  "continuous_on {r - epsilon<..<r + epsilon} f''" and raiz:  "f r = 0" and simple:  "f' r ~= 0"` **shows** `EX delta>0.  ALL x y.  |x - r| <= delta --> |y - r| <= delta --> x ~= y --> |paso_secante f x y - r| <= |y - r| / 2`

- `secante_5_convergencia` (test, dificultad 4, debilitada; clásica: `secante_5c_convergencia_local`). La secante converge a r desde x0 $\neq$ x1 en I.
  fixes `f f' f'' ::  "real => real" and r delta m M ::  real and x0 x1 ::  real` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx0: "|x0 - r| <= delta" and hx1: "|x1 - r| <= delta" and dist: "x0 ~= x1"` **shows** `(%n. secante f x0 x1 n) ----> r`

- `secante_5c_convergencia_local` (test, dificultad 5, clásica). Teorema clásico de convergencia local de la secante: existe $\delta > 0$ tal que converge desde todo x0 $\neq$ x1 en [r $-$ δ, r + δ].
  fixes `f f' f'' :: "real => real" and r epsilon :: real` **assumes** `eps: "0 < epsilon" and d1: "!!t. |t - r| < epsilon ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| < epsilon ==> (f' has_real_derivative f'' t) (at t)" and c2: "continuous_on {r - epsilon<..<r + epsilon} f''" and raiz: "f r = 0" and simple: "f' r ~= 0"` **shows** `EX delta>0. ALL x0 x1. |x0 - r| <= delta --> |x1 - r| <= delta --> x0 ~= x1 --> (%n. secante f x0 x1 n) ----> r`

- `secante_6_fibonacci` (test, dificultad 4, debilitada; clásica: `secante_6c_orden_phi`). Orden φ en forma explícita: M/(2m)|x_n $-$ r| $\leq (1/2)^{F(n+1)}$, con F de Fibonacci (F(n) ~ $\varphi^n/\sqrt{5}$).
  fixes `f f' f'' :: "real => real" and r delta m M :: real and x0 x1 :: real and n :: nat` **assumes** `d1: "!!t. |t - r| <= delta ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| <= delta ==> (f' has_real_derivative f'' t) (at t)" and raiz: "f r = 0" and m_pos: "0 < m" and cota1: "!!t. |t - r| <= delta ==> m <= |f' t|" and cota2: "!!t. |t - r| <= delta ==> |f'' t| <= M" and delta_pos: "0 < delta" and delta_chico: "M * delta <= m" and hx0: "|x0 - r| <= delta" and hx1: "|x1 - r| <= delta" and dist: "x0 ~= x1"` **shows** `M / (2 * m) * |secante f x0 x1 n - r| <= (1 / 2) ^ fibo (Suc n)`

- `secante_6c_orden_phi` (test, dificultad 5, clásica; análogo: `newton_6c_q_orden2`). Orden φ clásico (Q-orden): si f''(r) $\neq$ 0 y ningún iterado cae en r, $|e_{n+1}|/|e_n|^{\varphi} \to$ |f''(r)/(2f'(r))|$^{(\varphi-1)}$, con $\varphi = (1 + \sqrt{5})/2$.
  fixes `f f' f'' :: "real => real" and r epsilon :: real` **assumes** `eps: "0 < epsilon" and d1: "!!t. |t - r| < epsilon ==> (f has_real_derivative f' t) (at t)" and d2: "!!t. |t - r| < epsilon ==> (f' has_real_derivative f'' t) (at t)" and c2: "continuous_on {r - epsilon<..<r + epsilon} f''" and raiz: "f r = 0" and simple: "f' r ~= 0" and curva: "f'' r ~= 0"` **shows** `EX delta>0. ALL x0 x1. |x0 - r| <= delta --> |x1 - r| <= delta --> x0 ~= x1 --> (ALL n. secante f x0 x1 n ~= r) --> (%n. |secante f x0 x1 (Suc n) - r| / |secante f x0 x1 n - r| powr ((1 + sqrt 5) / 2)) ----> |f'' r / (2 * f' r)| powr ((1 + sqrt 5) / 2 - 1)`

A.6 Regula falsi (test)

Preludio: `definition punto_rf :: "(real => real) => real => real => real" where "punto_rf f l u = (l * f u - u * f l) / (f u - f l)" primrec regula_falsi :: "(real => real) => real => real => nat => real * real" where "regula_falsi f a b 0 = (a, b)" | "regula_falsi f a b (Suc n) = (case regula_falsi f a b n of (l, u) => (if f l * f (punto_rf f l u) <= 0 then (l, punto_rf f l u) else (punto_rf f l u, u)))"`

- `rf_1_punto` (test, dificultad 2, clásica). Si l $<$ u y f(l) $<$ 0 $\leq$ f(u), el punto de falsa posición cae en (l, u].
  fixes `f :: "real => real" and l u :: real` **assumes** `lu: "l < u" and fl: "f l < 0" and fu: "0 <= f u"` **shows** `l < punto_rf f l u & punto_rf f l u <= u`

- `rf_2_signo` (test, dificultad 2, clásica). Invariante de signo: f(l_n) $<$ 0 $\leq$ f(u_n).
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `fa: "f a < 0" and fb: "0 < f b"` **shows** `f (fst (regula_falsi f a b n)) < 0 & 0 <= f (snd (regula_falsi f a b n))`

- `rf_3_encaje` (test, dificultad 3, clásica). Horquillas encajadas no degeneradas: a $\leq$ l_n $<$ u_n $\leq$ b, l_n $\leq$ l_{n+1}, u_{n+1} $\leq$ u_n.
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `ab: "a < b" and fa: "f a < 0" and fb: "0 < f b"` **shows** `a <= fst (regula_falsi f a b n) & fst (regula_falsi f a b n) < snd (regula_falsi f a b n) & snd (regula_falsi f a b n) <= b & fst (regula_falsi f a b n) <= fst (regula_falsi f a b (Suc n)) & snd (regula_falsi f a b (Suc n)) <= snd (regula_falsi f a b n)`

- `rf_4_raiz` (test, dificultad 3, clásica). Cada horquilla [l_n, u_n] contiene una raíz.
  fixes `f :: "real => real" and a b :: real and n :: nat` **assumes** `ab: "a < b" and fa: "f a < 0" and fb: "0 < f b" and cont: "continuous_on {a..b} f"` **shows** `EX c. fst (regula_falsi f a b n) <= c & c <= snd (regula_falsi f a b n) & f c = 0`

- `rf_5_limites` (test, dificultad 3, clásica; arrastra hipótesis que no usa: la continuidad de f). Los dos extremos convergen (sucesiones monótonas acotadas).
  fixes `f :: "real => real" and a b :: real` **assumes** `ab: "a < b" and fa: "f a < 0" and fb: "0 < f b" and cont: "continuous_on {a..b} f"` **shows** `convergent (%n. fst (regula_falsi f a b n)) & convergent (%n. snd (regula_falsi f a b n))`

- `rf_6_convergencia` (test, dificultad 5, debilitada; clásica: `rf_6c_puntos_convergen`). Al menos un extremo converge a una raíz (el otro puede quedarse fijo).
  fixes `f :: "real => real" and a b :: real` **assumes** `ab: "a < b" and fa: "f a < 0" and fb: "0 < f b" and cont: "continuous_on {a..b} f"` **shows** `EX c. a <= c & c <= b & f c = 0 & ((%n. fst (regula_falsi f a b n)) ----> c | (%n. snd (regula_falsi f a b n)) ----> c)`

- `rf_6c_puntos_convergen` (test, dificultad 5, clásica). Forma clásica: si la raíz en [a, b] es única, los puntos de falsa posición c_n convergen a ella.
  fixes `f :: "real => real" and a b :: real` **assumes** `ab: "a < b" and fa: "f a < 0" and fb: "0 < f b" and cont: "continuous_on {a..b} f" and unica: "!!c d. c : {a..b} ==> d : {a..b} ==> f c = 0 ==> f d = 0 ==> c = d"` **shows** `EX c. a <= c & c <= b & f c = 0 & (%n. case regula_falsi f a b n of (l, u) => punto_rf f l u) ----> c`